

RedundaNet

E se una rete di storage non avesse un'azienda alle spalle, ma solo persone che si fidano?

Storage distribuito, cifrato sul tuo dispositivo, *molto* ridondante. Lo gestisce una comunità di volontari, non un'azienda.

Tahoe-LAFS · mesh VPN Tinc · identità GPG · un repo Git come source of truth · Docker Compose · Raspberry Pi, vecchi PC, un VPS da 4 €

redundanet.com · github.com/adefilippo83/redundanet



Alessandro De Filippo

Staff Site Reliability Engineer @ Celonis

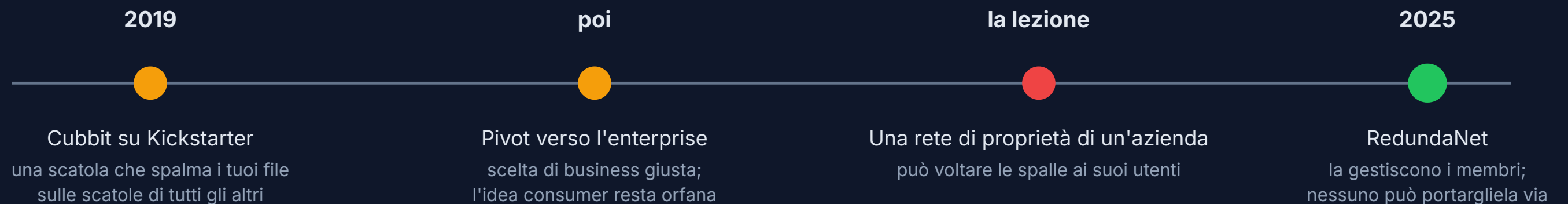
- Di mestiere tengo in piedi sistemi distribuiti in produzione e mi occupo di cosa succede quando si rompono.
- RedundaNet è il progetto del tempo libero: una rete vera, con persone vere dentro, non un esperimento sul portatile.
- Tutto quello che vedete oggi è software libero, GPL-3.

Email `alessandro@defilippo.me`

Sito `defilippo.me`

GitHub `github.com/adefilippo83`

Da dove viene: **un'idea rimasta orfana**



Molto distribuita (hardware qualunque, dietro i NAT di casa) · **molto ridondante** (erasure coding) · **si entra con un processo sociale**, non con una carta di credito.

Sei principi



Nessuno può leggere i tuoi dati

La cifratura avviene sul tuo dispositivo. Chi ospita vede solo ciphertext.



Qualunque nodo può morire

Erase coding, k-of-n configurabile.



Nessuna autorità centrale

Identità = la tua chiave GPG. Membership = un file in Git.



Hardware qualunque, dietro NAT

Router di casa, niente port forwarding. Una mesh VPN fa da relay.



Membri verificati, non anonimi

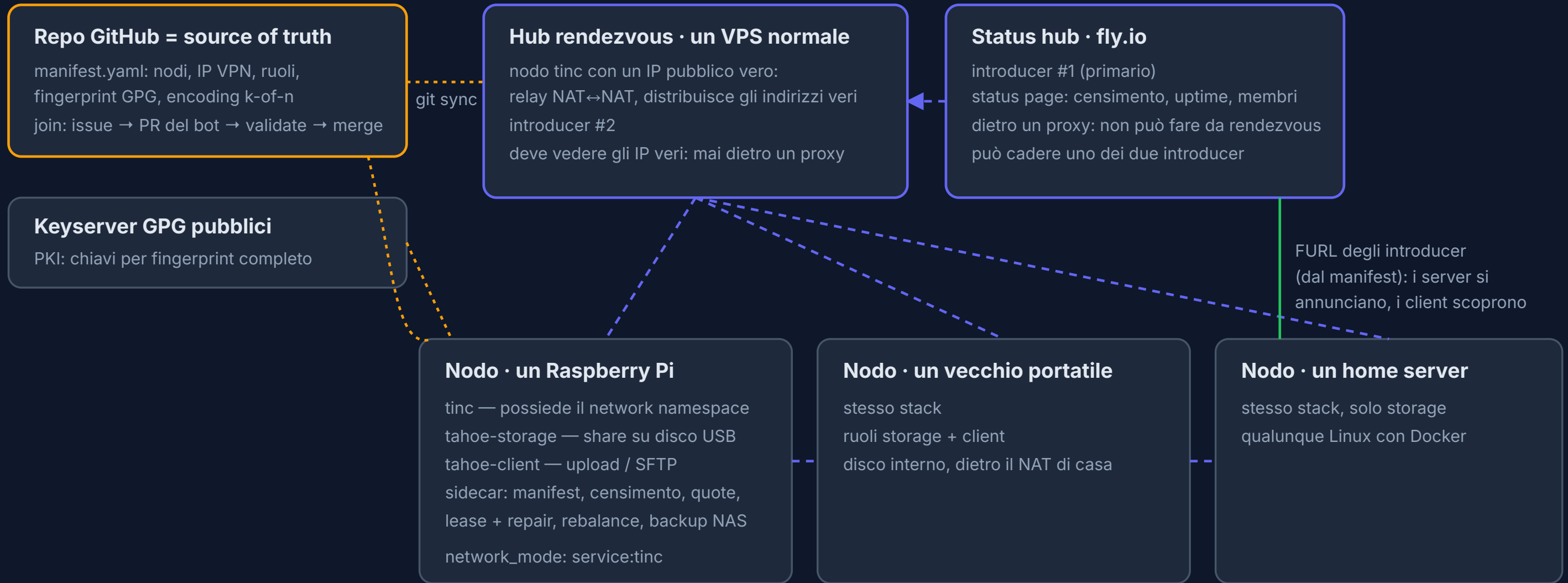
Persone approvate da persone. Alcune regole sono sulla fiducia, e lo diciamo.



Operatività noiosa

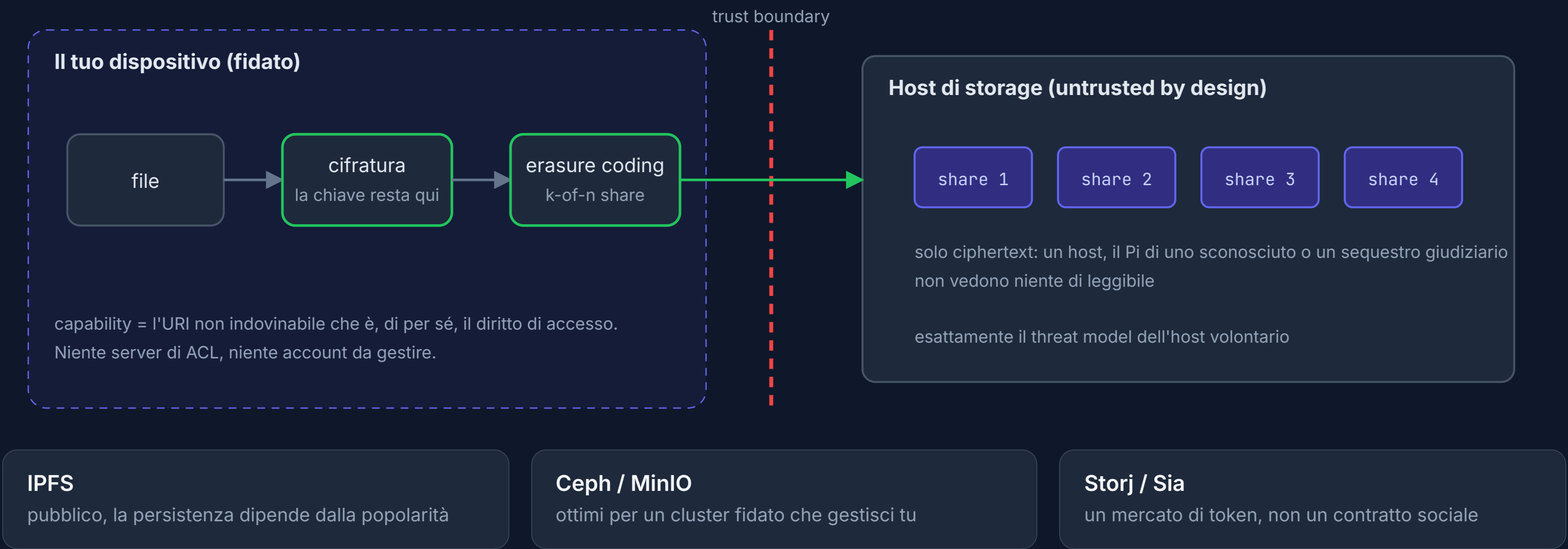
Una CLI, uno stack compose, aggiornamento automatico con rollback, una status page.

Architettura



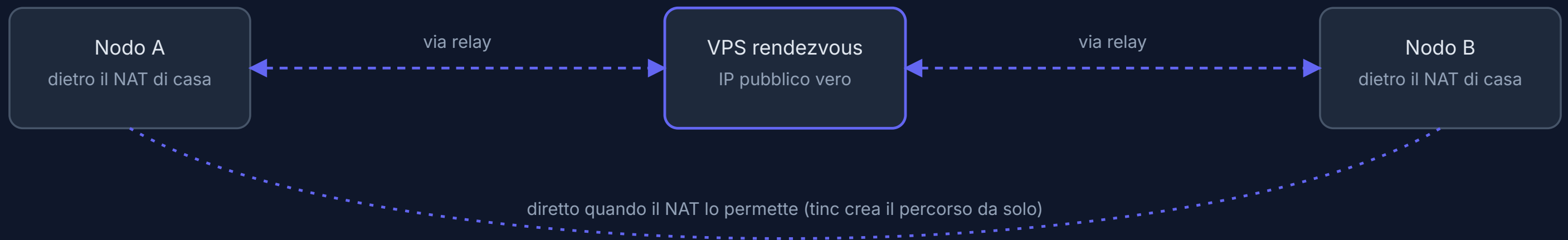
tratteggiato: mesh VPN tinc 10.100.0.0/16, ogni nodo con ogni nodo

Perché Tahoe-LAFS: il trust boundary

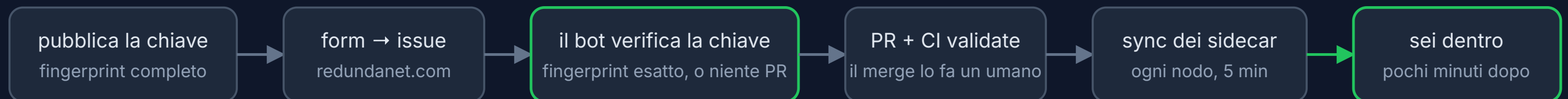


Connettività e identità: Tinc + GPG

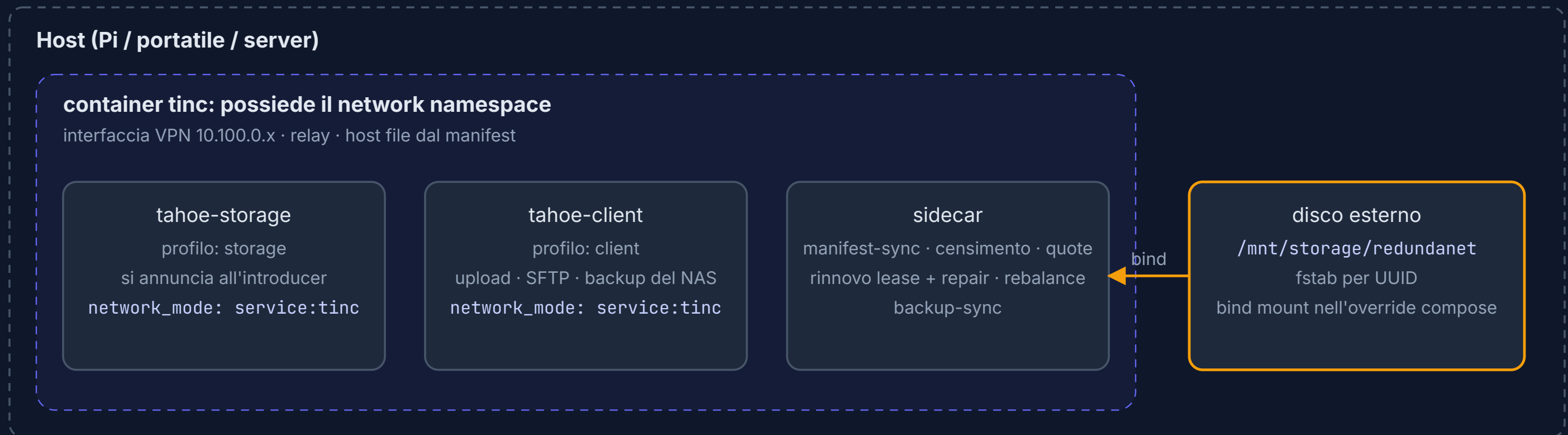
Un'identità sola: la chiave GPG del nodo è anche la sua chiave VPN



Join nella rete: fail closed



Dentro un nodo: **uno stack compose**, i ruoli sono profili



`docker compose -p redundanet --profile storage --profile client up -d` · i container Tahoe **entrano** nel namespace di tinc: niente port mapping da fare, e una trappola che vi faccio vedere tra poco.

LIVE DEMO 1

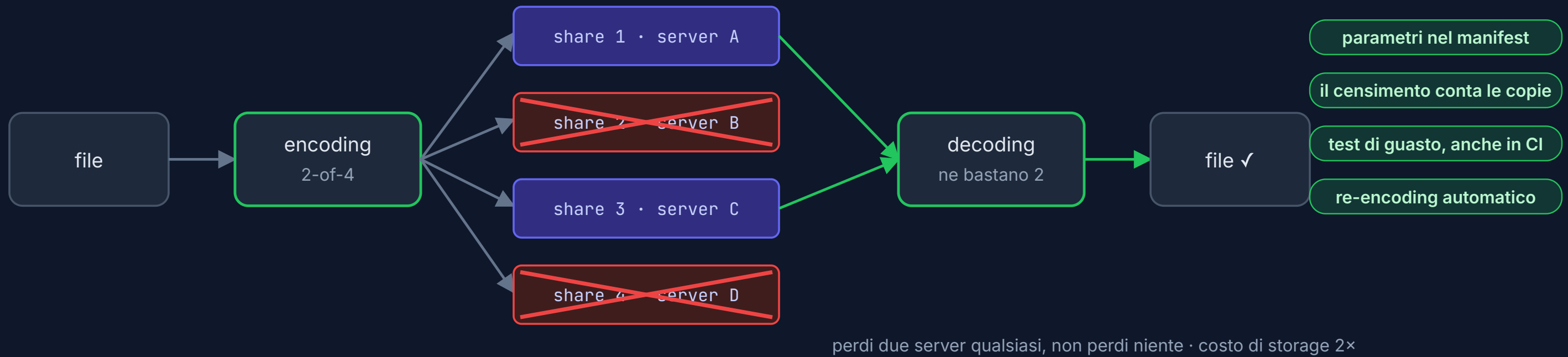
La rete, adesso

Apro `redundanet-hub.fly.dev`

- Nodi: raggiungibilità, RTT, uptime su 24h / 7d / 30d
- Server di storage annunciati agli introducer
- **Censimento della replica**: copie per oggetto rispetto al target, in continuo
- **Membri**: contributo, allocazione, utilizzo

```
# gli stessi dati della pagina (la forma; i valori li leggo dal vivo)
curl -s https://redundanet-hub.fly.dev/status.json \
  | jq '.replication'
{
  "objects_total": ...,
  "target_copies": 4,
  "fully_replicated": ...,
  "available_full": ...,
  "unreadable_now": 0
}
```

Ridondanza **misurata**, non presunta



La rete è partita come mirror **1-of-2** ed è passata a **2-of-4**: stesso costo 2×, sopravvive a due nodi. Il cambio è una riga nel manifest; ogni nodo la legge, i nuovi upload la usano subito e il rebalancer converte da solo i file vecchi.

Spengo un nodo, il file resta

```
# nodo A (client): carico il file, mi segno l'hash
sha256sum talk.pdf
redundanet storage upload talk.pdf home:talk.pdf

# nodo B (storage): lo butto giù
docker stop redundanet-tahoe-storage

# status page: un server in meno, il censimento mostra disponibilità ridotta
# nodo A: il download funziona ancora, dalle copie rimaste
redundanet storage download home:talk.pdf /tmp/back.pdf
sha256sum /tmp/back.pdf      # identico

docker start redundanet-tahoe-storage  # il censimento torna a replica completa
```

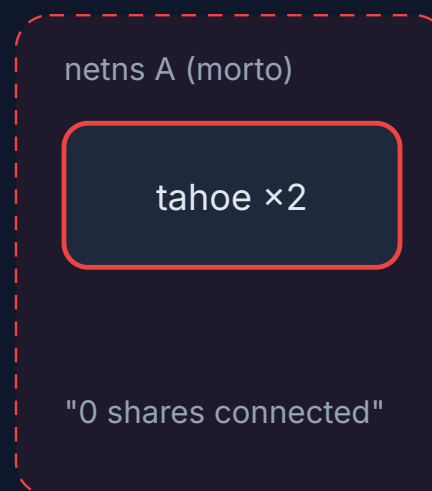
Correggi la **classe** di problema, non il caso singolo

Prima



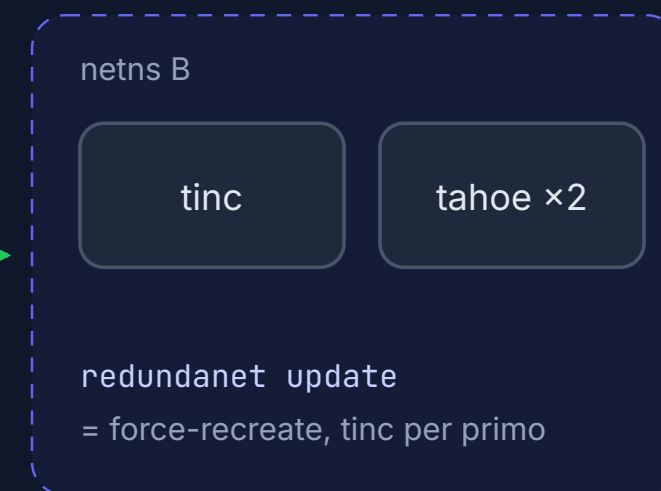
ricrea tinc

Dopo: la trappola



il fix

Sempre: tinc prima, poi tahoe



Il disco sparito

Un `-f` `compose.yml` esplicito disattiva l'auto-load dell'override di Docker; un update ha staccato in silenzio un disco da 1 TB. Corretto nella CLI, con un test che pretende due flag `-f`.

Il nodo nuovo che non parte

L'immagine per il Pi aveva Compose v1; un rifiuto nel manifest ha rotto un join per 11 minuti. Ora: Compose v2 pinnato e verificato in build, errori che dicono cosa fare al posto dei traceback.

Il rendezvous che mentiva

Un packet capture: ogni byte tra nodi passava dall'hub, in TCP. Il proxy di fly nascondeva gli indirizzi veri, e tinc si tiene il primo indirizzo che impara. Fix: un VPS da 4 € come rendezvous, una riga nel manifest.

Convivere con un upstream dormiente



SFTP non chiude mai le sessioni (bisect fatto: non è il client, non la rete, non Twisted) · niente accounting, quindi le quote stanno lato client
dipendenze transitive pinnate dopo che una release di pyOpenSSL 25 ha buttato giù tutti i container

La risposta

Piccoli workaround testati nelle nostre immagini · pinniamo le dipendenze che upstream non pinna · aggiriamo i frontend morti (web gateway, il pattern NAS) · documentiamo le stranezze dove l'utente le incontra.

L'assicurazione

Identità, membership e mesh non sanno che lo storage engine è Tahoe. L'engine si può sostituire; la rete no.

Hardening da un audit con 50 finding

Gate davanti alla source of truth



redundanet update



un :latest rotto non può lasciare un nodo a terra

Metodo: sei revisori per dimensione → verifica in contraddittorio di ogni finding → sei batch ordinati per blast radius, rilasciati in piccole release. Poi un secondo giro dall'interno: cosa può fare un *membro* malintenzionato? DoS sul relay, hijack degli indirizzi, storage server rogue. Classificati, in parte corretti, il resto in roadmap.

Testare un **sistema distribuito** su un portatile



Quello che i test **non** potevano intercettare: la trappola del netns, l'override perso, Compose v1 su un OS nuovo. Semantica dell'ambiente. Ognuno è diventato un unit test sulla costruzione dei comandi della CLI, perché il bug stava lì. E anche la build era un bug: la dipendenza Rust di Tahoe non ha wheel per ARM, quindi ogni release la compilava per un'ora sotto QEMU. Ora un'immagine base si ricostruisce solo quando cambia il lock, nativa per architettura, e una release ci mette minuti.

Usarla: un NAS di casa con una **macchina del tempo erasure-coded**



one-way per scelta: il sync bidirezionale vuol dire risolvere conflitti, e Magic Folder di upstream ci ha provato e ha rinunciato
l'ha proposto un membro; ha battuto la mia idea con FUSE perché lavora con i file immutabili invece che contro
l'usage meter conta un file invariato una volta sola tra tutti gli snapshot; il backup si ferma se un membro supera l'allocazione

Day-2 operations

```
# join: scrive .env, esporta la chiave GPG, stampa il comando esatto di avvio
redundanet network join --repo <repo del manifest> --name node-303c084b

# update sicuro: rileva via docker inspect, ricrea prima tinc, health check, rollback
redundanet update --check
redundanet update

# file e rete
redundanet storage upload foto.jpg home:foto.jpg
redundanet storage ls backups:Latest/
redundanet storage quota          # la mia allocazione e quanto uso sul grid
redundanet network peers
redundanet validate manifests/manifest.yaml
```

Tutto quello che serve a un nodo: /opt/redundanet/.env + un file di override per il disco. Gli errori dicono **cosa lanciare dopo**.

Cosa viene dopo (in ordine di rischio, non di divertimento)

Fatto: le promesse di ridondanza

- 2-of-4, convertito dal rebalancer
- due introducer, ne può cadere uno
- sweep settimanale di repair, rinnovo dei lease
- quote: allocazione per contributo, per membro
- un rendezvous che vede gli indirizzi veri
- release in minuti, non in un'ora

Adesso: hardening e pulizia

- hardening della mesh: subnet strette, cache del censimento, meno privilegi ai container
- contributo pesato per uptime e spazio libero
- pruning degli snapshot e policy di GC
- immagine dell'hub in CI, e2e notturni, alerting
- tooling per il disco del Pi

Poi: fiducia e la storia del fork

- certificati grid-manager: solo server verificati
- accounting lato server (spending pass)
- accesso da browser via web gateway di Tahoe
- il repo come template: fork, cambi un file, la tua rete, per esempio sui PC fermi in ufficio
- firma del manifest (rimandata apposta)

Cosa direi a me stesso all'inizio

- **Fai girare la tua rete dal primo giorno.** Ogni bug serio è arrivato dalla rete in produzione, e poi è diventato un test.
- **Una validazione fuori dal percorso reale è codice morto.**
- **Scegli in base al threat model, poi isola la scelta.**
- **Dichiara solo quello che gira davvero.**
- **Rendi leggibile il fallimento:** i fix migliori sono stati messaggi d'errore che dicono cosa lanciare dopo.
- **La topologia può mentire: misura i pacchetti.**
- **Una comunità progetta meglio di te.**

In numeri

2-of-4

possono morire due server qualsiasi; costo di storage 2×

4

tipi di hardware: Raspberry Pi, vecchio PC, server di casa, un VPS da 4 €

0

persone che possono leggere i tuoi file, host compresi

50

finding dell'audit, smistati in 6 batch

5 min

perché un cambio al manifest arrivi su ogni nodo; il censimento si aggiorna alla stessa cadenza

1

pull request per entrare

Uno storage in cui si entra con una pull request.

Molto distribuito. Molto ridondante. Nessuno in mezzo.

Codice github.com/adefilippo83/redundanet
Status redundanet-hub.fly.dev
Join redundanet.com/join.html
Contatti alessandro@defilippo.me · defilippo.me
Stack Python · Tahoe-LAFS · Tinc · GPG · Docker · GitHub Actions · fly.io

Domande che mi piacciono

Threat model e capability · perché non IPFS o Ceph · tinc vs WireGuard · perché un proxy non può fare da rendezvous · come funziona il censimento · cosa succede se muore un hub · quote senza accounting · cosa può fare un membro malintenzionato